

The Complete Data Analyst & Data Science Roadmap

A premium, company-grade handbook from beginner/intermediate to professional Data Analyst — with a runway into Data Science.

Compiled 2026-07-31

Table of Contents

- 01 · Introduction
- 02 · How Companies Work
- 03 · Databases
- 04 · SQL — The Complete Course
- 05 · Python
- 06 · Power BI
- 07 · Professional Projects
- 08 · Machine Learning
- 09 · Portfolio & Career

01 • Introduction

Part 1 of *The Complete Data Analyst & Data Science Roadmap*

 **What you'll be able to do after this chapter** Explain, in an interview or to a non-technical stakeholder, exactly what a Data Analyst does differently from a Data Scientist, a Data Engineer, and an Analytics Engineer — and pick which of those career paths to aim for, with a realistic sense of skills, salary, and daily work.

1.1 What is Data?

Forget the textbook definition for a second. Inside a company, **data is exhaust**. Every action a user takes, every system that runs, every transaction that clears — leaves a trace. A click, a payment, a delivery scan, a support ticket, a sensor reading.

Nobody sets out to "collect data." Data is a *byproduct* of the business running. Your job, in every role this handbook covers, is to take that byproduct and turn it into something the business can act on.

flowchart LR

```
A[User does something] → B[App records an event]
B → C[Event lands in a database]
C → D["Someone asks a question\n('why did signups drop?')"]
D → E[You turn raw records into an answer]
E → F[Business makes a decision]
```

 **Real company example** At **Spotify**, playing a song, skipping it after 4 seconds, or adding it to a playlist are all events. None of them were generated "for analytics" — they're just what the app does when you use it. The Data team's entire job starts *after* that event already exists.

Data on its own is worthless. It becomes valuable the moment someone turns it into a decision — that transformation, from raw event to decision, is what this entire handbook teaches, one layer at a time:

```
Users → Applications → Operational Database → ETL/ELT → Data Warehouse
      → SQL → Python → Power BI → Business Decision → Machine Learning → Deployment
```

You'll see this exact pipeline again in [Chapter 02](#), where each arrow gets its own deep dive.

1.2 The six roles, and how they actually differ

Job titles in data are inconsistent across companies — a "Data Analyst" at one company does what a "Data Scientist" does at another. But the *work itself* clusters into six recognizable jobs. Here's how to tell them apart by what they actually spend their day doing, not by their title.

Role	Core question they answer	Primary tools	Output
Business Intelligence (BI) Analyst	"What happened, and how do I make it visible to everyone?"	SQL, Power BI / Tableau	Dashboards, recurring reports
Data Analyst	"Why did it happen, and what should we do?"	SQL, Excel, Python, Power BI	Ad-hoc analysis, insights, decks
Data Scientist	"What will happen, and how confident are we?"	Python, statistics, ML	Models, experiments, predictions
Data Engineer	"How do we move and store data reliably at scale?"	SQL, Python, Spark, Airflow, cloud	Pipelines, warehouses
Analytics Engineer	"How do we make clean, trusted data models everyone can query?"	SQL, dbt, warehouse tooling	Data models, transformation layer
ML Engineer	"How do we get a model running reliably in production?"	Python, APIs, Docker, cloud, MLOps	Deployed, monitored models

✦ **Callout — the fastest way to tell two roles apart** Ask: "*What do they hand off, and to whom?*"

- A **BI Analyst** hands a dashboard to an executive. - A **Data Analyst** hands an insight to a decision-maker. - A **Data Scientist** hands a model or a tested hypothesis to a Data Analyst or a product team. - A **Data Engineer** hands clean, reliable tables to everyone above. - An **Analytics Engineer** hands trusted, documented data *models* (not raw tables) to Analysts and Scientists. - An **ML Engineer** hands a running, monitored service to the product itself.

How this looks inside a real company

```

flowchart TB
    subgraph Engineering
        DE[Data Engineer\nbuilds pipelines & warehouse]
        AE[Analytics Engineer\nmodels the warehouse into\nclean, reusable tables]
    end
    subgraph Analysis
        BI[BI Analyst\ndashboards & reporting]
        DA[Data Analyst\nad-hoc insight & investigation]
    end
    subgraph Science
        DS[Data Scientist\nmodeling & experimentation]
        ME[ML Engineer\nproductionizing models]
    end

```

```

end
DE → AE → BI
AE → DA
AE → DS
DS → MLE
BI → Biz[Business Decision]
DA → Biz
MLE → Prod[Live Product Feature]

```

 **Real company example** At **Mercado Libre**, when a Product Manager asks "why is cart abandonment up in Brazil?", a **Data Analyst** answers it with SQL and a deck within days. If the answer becomes "we should build a model to predict who's about to abandon their cart," that becomes a **Data Scientist** project — and if it ships as a live discount-trigger in the app, an **ML Engineer** puts it into production and keeps it running.

1.3 Daily responsibilities, role by role

BI Analyst

- Maintain and extend recurring dashboards (revenue, retention, ops metrics)
- Own metric definitions so "revenue" means the same thing to everyone
- Triage "the dashboard looks wrong" tickets
- Present weekly/monthly business reviews

Data Analyst

- Get pulled into a Slack thread with a vague question ("are we losing money on this promo?") and turn it into a scoped analysis
- Write SQL against the warehouse, sanity-check the numbers, visualize them
- Present findings with a clear recommendation, not just a chart
- Partner with Product/Marketing/Finance stakeholders directly

Data Scientist

- Frame a business problem as a modeling problem (classification, regression, ranking, etc.)
- Explore data, engineer features, train and validate models
- Design and read A/B test results
- Communicate uncertainty and trade-offs to non-technical stakeholders

Data Engineer

- Build and maintain ETL/ELT pipelines

- Design warehouse schemas for reliability and query performance
- Own data quality checks, alerting, and pipeline SLAs
- Manage cloud infrastructure costs and scaling

Analytics Engineer

- Write and test transformation logic (commonly in dbt) on top of raw warehouse data
- Document tables and columns so self-serve analytics actually works
- Own the "single source of truth" layer between raw data and everyone who queries it
- Bridge the gap between Data Engineering and Analysis

ML Engineer

- Take a Data Scientist's notebook and turn it into a reliable service
- Build training/inference pipelines, APIs, monitoring, and retraining triggers
- Manage model versioning and rollback
- Own latency, cost, and uptime of ML in production

1.4 Career paths

None of these roles are dead ends — they're a graph, not a ladder. The most common entry point for this handbook's audience is **Data Analyst**, and the two dominant paths from there are:

```

flowchart LR
    Start([Junior Data Analyst]) --> DA[Data Analyst]
    DA --> SDA[Senior Data Analyst]
    SDA --> AE[Analytics Engineer]
    SDA --> DS[Data Scientist]
    AE --> DEng[Data Engineer]
    DS --> MLE[ML Engineer]
    SDA --> LeadA[Analytics Manager]
    DS --> LeadD[Data Science Manager]
  
```

✅ **Best practice** Don't try to pick your final destination on day one. Start as a Data Analyst, get fluent in SQL and business context, and let the work itself tell you whether you enjoy the "build reliable systems" side (→ Analytics/Data Engineering) or the "model uncertainty" side (→ Data Science) more.

⚠️ **Common mistake** Jumping straight to Machine Learning tutorials without ever having queried a real, messy, undocumented production database. Companies don't hire ML Engineers who can't handle dirty data — they hire Analysts who grew into it. [Chapter 08](#) assumes you've been through [Chapters 03–05](#) first for exactly this reason.

1.5 Salary ranges (indicative benchmarks)

Salaries vary enormously by country, company size, and seniority — treat these as **directional**, not contractual.

Role	LatAm (remote, USD/yr)	US (onsite/remote, USD/yr)	EU (onsite/remote, EUR/yr)
BI Analyst	\$18k – \$40k	\$65k – \$100k	€35k – €60k
Data Analyst	\$20k – \$45k	\$70k – \$110k	€38k – €65k
Senior Data Analyst	\$35k – \$65k	\$100k – \$140k	€55k – €85k
Data Scientist	\$30k – \$60k	\$110k – \$165k	€55k – €95k
Data Engineer	\$30k – \$65k	\$115k – \$170k	€55k – €95k
Analytics Engineer	\$30k – \$60k	\$110k – \$155k	€55k – €90k
ML Engineer	\$35k – \$75k	\$130k – \$190k	€65k – €110k

✦ **Callout** Total compensation (equity, bonuses) at large tech companies (Google, Amazon, Microsoft, Stripe) can push the upper end of the US range 30–60% higher. LatAm ranges above assume **remote work for a foreign company** — local-market-only salaries are typically 40–60% lower.

1.6 Required skills matrix

Skill	BI Analyst	Data Analyst	Data Scientist	Data Engineer	Analytics Engineer	ML Engineer
SQL	●●●	●●●	●●○	●●●	●●●	●●○
Excel / Spreadsheets	●●●	●●●	●○○	●○○	●○○	○○○
Python	●○○	●●○	●●●	●●●	●●○	●●●
Statistics	●○○	●●○	●●●	●○○	●○○	●●○
Data Visualization (Power BI/Tableau)	●●●	●●●	●○○	○○○	●○○	○○○
Data Modeling / Warehousing	●●○	●●○	●○○	●●●	●●●	●○○

Skill	BI Analyst	Data Analyst	Data Scientist	Data Engineer	Analytics Engineer	ML Engineer
Machine Learning	○○○	●○○	●●●	○○○	○○○	●●●
Software Engineering / APIs	○○○	○○○	●○○	●●●	●●○	●●●
Business Communication	●●●	●●●	●●○	●○○	●●○	●○○
Cloud Platforms (AWS/Azure/GCP)	●○○	●○○	●●○	●●●	●●○	●●●

●●● core to the role · ●●○ important · ●○○ useful · ○○○ rarely needed

This handbook builds these skills in the order most companies actually need them: **Databases** → **SQL** → **Python** → **Power BI** → **Projects** → **Machine Learning** → **Portfolio**, matching the pipeline from §1.1.

1.7 Recommended books

Book	Author	Best for
<i>SQL for Data Analysis</i>	Cathy Tanimura	SQL fundamentals with real analytical patterns
<i>Storytelling with Data</i>	Cole Nussbaumer Knaflic	Turning charts into decisions people act on
<i>Python for Data Analysis</i>	Wes McKinney (creator of Pandas)	The definitive Pandas reference
<i>The Data Warehouse Toolkit</i>	Ralph Kimball	Star schema, dimensional modeling — foundational for Ch.03
<i>Practical Statistics for Data Scientists</i>	Peter Bruce, Andrew Bruce, Peter Gedeck	Statistics with a practitioner's lens, not a professor's
<i>Designing Data-Intensive Applications</i>	Martin Kleppmann	For when you want to understand what Data Engineers actually build
<i>Hands-On Machine Learning with Scikit-Learn, Keras & TensorFlow</i>	Aurélien Géron	The standard practical ML reference

1.8 Recommended YouTube channels

Channel	Focus
Luke Barousse	Data Analyst career advice, real job-market insight

Channel	Focus
Alex The Analyst	SQL, Power BI, and Data Analyst fundamentals from scratch
StatQuest with Josh Starmer	Statistics and ML concepts explained visually, no fluff
Seattle Data Guy	Data Engineering and career strategy from an industry practitioner
Ken Jee	Data Science career paths and portfolio-building
Corey Schafer	Deep, correct Python fundamentals
Guy in a Cube	Power BI straight from a former Microsoft Power BI team member

Chapter Summary

- Data has no value until it's turned into a decision — that transformation is the job, across every role in this field.
- BI Analyst, Data Analyst, Data Scientist, Data Engineer, Analytics Engineer, and ML Engineer are six distinct jobs distinguished by *what they hand off and to whom*, not by title alone.
- Most people entering this field should start as a **Data Analyst** and branch toward Data Science or Analytics/Data Engineering based on what part of the job they enjoy.
- Skills compound in a specific order — this handbook follows that order starting in [Chapter 02](#).

Next: [02. How Companies Work](#) → — how data is actually generated and stored before anyone gets to analyze it.

02 · How Companies Work

Part 2 of *The Complete Data Analyst & Data Science Roadmap* · Previous: [01. Introduction](#)

 **What you'll be able to do after this chapter** Draw and explain the real data architecture of a modern company — from a user tapping a button to an executive seeing a number on a dashboard — and know when a company reaches for a Data Warehouse vs. a Data Lake vs. a Lakehouse, and batch vs. streaming.

2.1 How data is generated

Every company runs on **applications** — the mobile app, the website, the point-of-sale system, the internal admin tool. Those applications need to remember things *right now*: did the payment go through, is the item still in stock, is the user logged in. That immediate, transactional memory lives in an **operational database** (also called OLTP — see [Chapter 03](#)).

flowchart LR

```
U[User] -->|clicks / buys / signs up| App[Application]
App -->|reads/writes rows| DB[(Operational Database\nOLTP)]
App2[Backend services] -->|writes events| DB
```

 **Real company example** When you add an item to your cart on **Mercado Libre**, that write goes straight into an operational database optimized for *speed and correctness of a single transaction* — not for answering "what were our top 10 categories last month?" That question needs a completely different system, which is where the rest of this chapter comes in.

2.2 Operational Database → Data Warehouse

Operational databases are built to handle thousands of small, fast read/write transactions. They are **bad** at answering big analytical questions ("total revenue by region, by month, for the last 3 years") because that kind of query scans huge amounts of data — and running that against the live database would slow down the app for real users.

So companies copy the data out into a second system, purpose-built for analysis: the **Data Warehouse**.

flowchart LR

```

DB[(Operational DB\nOLTP)] -->|ETL / ELT| DW[(Data Warehouse\nOLAP)]
DW --> SQL[SQL Analysis]
DW --> BI[Power BI / Tableau]
DW --> DS[Data Science / ML]

```

The process that moves and reshapes the data between these two systems is called **ETL** or **ELT**:

	ETL (Extract, Transform, Load)	ELT (Extract, Load, Transform)
Order	Transform <i>before</i> loading	Transform <i>after</i> loading
Where transformation happens	A separate processing engine	Inside the warehouse itself
Common today because	Warehouses are now powerful enough to transform data at scale	Cloud warehouses (Snowflake, BigQuery, Redshift) are cheap to run big transformations in
Typical tooling	Informatica, legacy SSIS, custom scripts	Airbyte/Fivetran (extract+load) + dbt (transform)

✅ **Best practice** Modern cloud-first companies default to **ELT**: land raw data in the warehouse first, then transform it with SQL/dbt where it's easy to test, version, and re-run. This is also the layer where **Analytics Engineers** live (see [01.2](#)).

⚠️ **Common mistake** Querying the operational (production) database directly for analytics. This is how junior analysts accidentally slow down — or crash — the live app. Always analyze from the warehouse or a read replica, never the primary production database.

2.3 Data Warehouse vs. Data Lake vs. Lakehouse

	Data Warehouse	Data Lake	Lakehouse
Data shape	Structured (tables, rows, columns)	Any shape — structured, semi-structured, raw files, images, logs	Structured + unstructured, unified
Schema	Enforced on write (schema-on-write)	Enforced on read (schema-on-read)	Both, depending on layer
Best for	BI dashboards, SQL analytics	ML training data, raw logs, data science exploration	Everything — one platform for both
Examples	Snowflake, BigQuery, Redshift	S3 + Hive/Glue, Azure Data Lake	Databricks, Snowflake (Iceberg), BigQuery (unified)

 **Real company example** A company like **Netflix** dumps raw viewing-event logs into a **Data Lake** because the volume and variety is enormous and not every team needs it structured. The BI team then builds a curated, structured layer on top — either a classic **Data Warehouse** or, increasingly, a **Lakehouse** that lets both SQL analysts and ML teams work off the same underlying storage without duplicating data.

2.4 Batch vs. Streaming

	Batch Processing	Streaming
Data moves	On a schedule (hourly, nightly)	Continuously, as events happen
Latency	Minutes to hours	Milliseconds to seconds
Typical tools	Airflow + SQL/dbt jobs, scheduled Python scripts	Kafka, Kinesis, Flink, Spark Streaming
Typical use case	Daily revenue dashboard, monthly reports	Fraud detection, real-time recommendations, live inventory

 **Callout** Most **Data Analyst** work happens on top of **batch-processed** data — a dashboard that refreshes every morning is completely normal and expected. Streaming is a Data/ML Engineering concern you'll encounter, but rarely build yourself as an analyst.

2.5 Cloud platforms: AWS, Azure, Google Cloud

You don't need to master any cloud provider to start as a Data Analyst, but you will constantly hear these names — know what they're for.

Provider	Warehouse	Storage (Lake)	BI Tool	Notes
AWS	Redshift	S3	QuickSight (rare)	Most common cloud overall; often paired with Power BI or Tableau
Azure	Synapse Analytics	Azure Data Lake Storage	Power BI (native)	Common in enterprises already on Microsoft stack
Google Cloud	BigQuery	Cloud Storage	Looker / Looker Studio	Popular with tech-native, high-growth companies

✓ **Best practice** When you see a job posting say "SQL + BigQuery" or "SQL + Snowflake," don't panic — the SQL you learn in [Chapter 04](#) is 90% transferable. The differences are mostly dialect syntax and how you connect, not how you think about queries.

2.6 The full picture

```

flowchart TB
    U[Users] --> App[Applications]
    App --> OLTP[(Operational Database\<br>OLTP)]
    OLTP --> ELT[ELT]
    ELT --> DW[(Data Warehouse)]
    Lake[(Data Lake\<br>raw/unstructured)] --> DW
    DW --> SQLA[SQL]
    SQLA --> PY[Python]
    PY --> PBI[Power BI]
    PBI --> Dec[Business Decision]
    PY --> ML[Machine Learning]
    ML --> Dep[Deployment]
    Dep --> App
  
```

This is the diagram to keep in your head for the rest of the handbook — every remaining chapter drops into one node of it: [Databases](#) and [SQL](#) live in the warehouse layer, [Python](#) and [Power BI](#) sit on top of it, and [Machine Learning](#) closes the loop back into the product.

Chapter Summary

- Applications write to fast, transactional **operational databases** — never analyze directly against them.
- **ETL/ELT** moves and reshapes that data into a **Data Warehouse** built for analysis; modern companies favor ELT with SQL/dbt.
- **Data Lakes** hold raw/unstructured data at scale; **Lakehouses** try to unify both worlds.
- Most analyst work runs on **batch** data; **streaming** is a Data/ML Engineering concern.
- AWS, Azure, and GCP each offer a warehouse, a lake, and (for Azure) a native BI tool — the SQL skills you'll build in Chapter 04 transfer across all of them.

Next: [03. Databases](#) → — what's actually inside that warehouse box, and how it's structured so SQL can be fast.

03 • Databases

Part 3 of *The Complete Data Analyst & Data Science Roadmap* · Previous: [02. How Companies Work](#)

💡 **What you'll be able to do after this chapter** Read and design a relational schema, explain normalization, and design a Star Schema — the skill that makes everything in [Chapter 04 \(SQL\)](#) click into place.

3.1 Relational databases, in one picture

A relational database stores data in **tables** (rows × columns), and tables relate to each other through **keys**.

```
erDiagram
    CUSTOMERS ||--o{ ORDERS : places
    ORDERS ||--}| ORDER_ITEMS : contains
    PRODUCTS ||--o{ ORDER_ITEMS : "appears in"

    CUSTOMERS {
        int customer_id PK
        string name
        string email
        date signup_date
    }
    ORDERS {
        int order_id PK
        int customer_id FK
        date order_date
        string status
    }
    ORDER_ITEMS {
        int order_item_id PK
        int order_id FK
        int product_id FK
        int quantity
        decimal unit_price
    }
    PRODUCTS {
        int product_id PK
        string product_name
        string category
    }
```

```
decimal price
}
```

Primary Keys & Foreign Keys

- **Primary Key (PK):** the column (or set of columns) that uniquely identifies each row in a table. `customer_id` in `CUSTOMERS`.
- **Foreign Key (FK):** a column in one table that points to a Primary Key in another, creating a relationship. `customer_id` in `ORDERS` points back to `CUSTOMERS`.

 **Real company example** At any e-commerce company, `orders.customer_id` is a foreign key into `customers.customer_id`. This is *how* the system knows which customer placed which order — and it's the exact relationship you'll `JOIN` on constantly in [Chapter 04](#).

Indexes

An **index** is a lookup structure that lets the database find rows without scanning the entire table — the same idea as a book's index letting you skip straight to a page instead of reading cover to cover.

 **Best practice** Primary Keys are indexed automatically. As an analyst, the index decision that affects you most is: columns you constantly filter or join on (`WHERE customer_id = ...` , `JOIN ... ON order_id`) benefit from an index. You won't usually create indexes yourself, but you'll diagnose slow queries by asking "is this missing one?" — covered in [04.7 Performance & Optimization](#).

3.2 Normalization

Normalization is the process of organizing tables to eliminate duplicate data and inconsistency.

Form	Rule	Fixes
1NF	Every column holds a single value (no comma-separated lists in a cell)	Repeating groups in one field
2NF	1NF + every non-key column depends on the <i>whole</i> primary key	Partial dependency in composite keys
3NF	2NF + no non-key column depends on another non-key column	Transitive dependency (e.g., storing <code>city</code> and <code>country</code> when <code>city</code> already determines <code>country</code>)

⚠️ **Common mistake** Storing `customer_name` on every single row of the `ORDERS` table instead of just `customer_id`. Now if a customer changes their name, you have to update it in a thousand places — and if you miss one, your data is now inconsistent. Normalization exists to prevent exactly this.

🏢 **Real company example** Operational databases (the OLTP systems behind the app) are almost always heavily normalized — it keeps writes fast and consistent. But the *warehouse* often intentionally **denormalizes** parts of that structure, because analysts value query simplicity and read speed over write efficiency. That trade-off is exactly what a Star Schema is for.

3.3 Star Schema vs. Snowflake Schema

Once data lands in the warehouse, it's typically remodeled around **Fact tables** and **Dimension tables**.

- **Fact table:** the "what happened" — one row per event/transaction, mostly numbers (measures) and foreign keys.
- **Dimension table:** the "who/what/where/when" — descriptive attributes you slice and filter by.

```
erDiagram
    FACT_SALES ||--o{ DIM_DATE : "occurred on"
    FACT_SALES ||--o{ DIM_CUSTOMER : "sold to"
    FACT_SALES ||--o{ DIM_PRODUCT : "of product"
    FACT_SALES ||--o{ DIM_STORE : "sold at"

    FACT_SALES {
        int sale_id PK
        int date_id FK
        int customer_id FK
        int product_id FK
        int store_id FK
        decimal revenue
        int quantity
    }
    DIM_DATE { int date_id PK, date full_date, int year, int month, string weekday }
    DIM_CUSTOMER { int customer_id PK, string name, string segment }
    DIM_PRODUCT { int product_id PK, string name, string category }
    DIM_STORE { int store_id PK, string city, string region }
```

This is a **Star Schema**: one central fact table surrounded directly by flat dimension tables. It's called that because the diagram looks like a star.

A **Snowflake Schema** takes it further by normalizing the dimensions themselves — e.g., splitting `DIM_PRODUCT` into `DIM_PRODUCT` → `DIM_CATEGORY` → `DIM_DEPARTMENT`.

	Star Schema	Snowflake Schema
Dimensions	Denormalized (flat)	Normalized (split into sub-tables)
Query complexity	Simpler — fewer joins	More joins required
Query speed	Generally faster	Can be slower due to extra joins
Storage	Slightly more redundant	More storage-efficient
When companies choose it	Default choice for BI/reporting — favors analyst simplicity	Very large dimension tables, or strict data governance needs

✅ **Best practice** As a Data Analyst, default to expecting a **Star Schema** in a well-built warehouse — it's what **Analytics Engineers** optimize for, because it's what makes Power BI and SQL fast and understandable. If you're handed a Snowflake Schema, expect more joins in your everyday queries.

3.4 OLTP vs. OLAP

	OLTP (Online Transaction Processing)	OLAP (Online Analytical Processing)
Purpose	Run the business, transaction by transaction	Analyze the business, in aggregate
Example system	The app's production database	The Data Warehouse
Query pattern	Short, simple, high-frequency (<code>INSERT</code> , single-row <code>UPDATE</code>)	Long, complex, aggregating over millions of rows
Schema	Normalized	Star/Snowflake (partially denormalized)
Who touches it	The application itself	Analysts, Scientists, BI tools

This is the same distinction from [02.2](#) — now you know *why* the two systems are shaped so differently: OLTP is optimized for **normalized, fast writes**; OLAP is optimized for **denormalized, fast aggregate reads**.

3.5 SQL preview

You don't need full SQL fluency yet ([Chapter 04](#) is the complete course), but here's how the schema above turns into a real question:

```
-- "Total revenue by product category, last quarter"
SELECT
    p.category,
    SUM(f.revenue) AS total_revenue
FROM fact_sales f
JOIN dim_product p ON f.product_id = p.product_id
JOIN dim_date d ON f.date_id = d.date_id
WHERE d.full_date ≥ '2026-04-01' AND d.full_date < '2026-07-01'
GROUP BY p.category
ORDER BY total_revenue DESC;
```

Notice how directly this maps to the Star Schema: one `JOIN` per dimension you need, one `GROUP BY` for the level of detail you want. That's the entire point of designing it this way.

Chapter Summary

- Relational databases connect tables through **Primary Keys** and **Foreign Keys**; **indexes** make lookups fast.
- **Normalization** (1NF–3NF) removes duplication and inconsistency — critical for OLTP systems.
- Warehouses remodel data into **Fact** and **Dimension** tables, typically as a **Star Schema** (simpler, faster) or a **Snowflake Schema** (more normalized, more joins).
- **OLTP** runs the business transaction-by-transaction; **OLAP** analyzes it in aggregate — different shapes for different jobs.

Next: [04. SQL →](#) — the complete SQL course, built directly on the schema concepts from this chapter.

04 · SQL — The Complete Course

Part 4 of *The Complete Data Analyst & Data Science Roadmap* · Previous: [03. Databases](#)

 **What you'll be able to do after this chapter** Write real analytical SQL end to end — filtering, aggregating, joining, windowing, and optimizing — against the Star Schema from [Chapter 03](#). This is the single highest-leverage skill in the whole handbook.

All examples use the `fact_sales` / `dim_product` / `dim_customer` / `dim_date` / `dim_store` schema introduced in [03.3](#). Solutions for every exercise live in [../exercises/04_SQL/solutions/](#).

4.1 SELECT, WHERE, ORDER BY

```
SELECT product_name, category, price
FROM dim_product
WHERE category = 'Electronics'
ORDER BY price DESC;
```

 **Business case:** Marketing asks for "every Electronics product priced above \$50, most expensive first" to plan a promo.

Exercise: List all customers who signed up in 2025, ordered by signup date.

► Solution

4.2 GROUP BY, HAVING

`GROUP BY` collapses rows into groups; `HAVING` filters *after* aggregation (`WHERE` filters *before*).

```
SELECT category, SUM(revenue) AS total_revenue
FROM fact_sales f JOIN dim_product p ON f.product_id = p.product_id
GROUP BY category
HAVING SUM(revenue) > 100000;
```

 **Business case:** Finance wants only categories that generated over \$100k — small categories are noise for this review.

Exercise: Find stores with more than 500 total orders.

► Solution

4.3 JOIN, UNION

JOIN type	Returns
INNER JOIN	Only rows matching in both tables
LEFT JOIN	All rows from the left table, matched rows from the right (NULL if no match)
RIGHT JOIN	Mirror of LEFT JOIN
FULL OUTER JOIN	All rows from both, matched where possible

```
-- Customers with no orders yet (classic LEFT JOIN use case)
SELECT c.customer_id, c.name
FROM dim_customer c
LEFT JOIN fact_sales f ON c.customer_id = f.customer_id
WHERE f.customer_id IS NULL;
```

UNION stacks result sets vertically (and removes duplicates; **UNION ALL** keeps them — and is faster).

 **Business case:** Growth wants a "never purchased" list for a first-order discount campaign — exactly the `LEFT JOIN ... WHERE ... IS NULL` pattern above.

Exercise: Combine a list of online orders and in-store orders into one result set, keeping duplicates.

► Solution

4.4 CTEs & Subqueries

A **CTE** (Common Table Expression, **WITH**) names a subquery so you can build a query in readable steps.

```
WITH monthly_revenue AS (
  SELECT DATE_TRUNC('month', d.full_date) AS month, SUM(f.revenue) AS revenue
  FROM fact_sales f JOIN dim_date d ON f.date_id = d.date_id
  GROUP BY 1
```

```

)
SELECT month, revenue,
       revenue - LAG(revenue) OVER (ORDER BY month) AS mom_change
FROM monthly_revenue
ORDER BY month;

```

✅ **Best practice** Prefer CTEs over deeply nested subqueries — they read top-to-bottom like a narrative, and every analyst on your team can debug them piece by piece.

Exercise: Find the top-spending customer per region using a CTE.

► Solution

4.5 Views & Stored Procedures

- **View:** a saved query you can `SELECT` from like a table — great for hiding complexity from other analysts.
- **Stored Procedure:** a saved, parameterized block of SQL (and logic) you `CALL` — used more by engineers than analysts, but you should recognize one when you see it.

```

CREATE VIEW vw_monthly_category_revenue AS
SELECT DATE_TRUNC('month', d.full_date) AS month, p.category, SUM(f.revenue) AS revenue
FROM fact_sales f
JOIN dim_date d ON f.date_id = d.date_id
JOIN dim_product p ON f.product_id = p.product_id
GROUP BY 1, 2;

```

📊 **Business case:** Instead of every analyst re-writing the same 3-table join every week for the exec dashboard, an **Analytics Engineer** exposes it as a view — this is the "single source of truth" idea from [01.2](#).

4.6 Window Functions & Ranking

Window functions compute across a set of rows *related to the current row*, without collapsing them like `GROUP BY` does.

Function	Use
<code>ROW_NUMBER()</code>	Unique sequential number per row

Function	Use
<code>RANK()</code> / <code>DENSE_RANK()</code>	Ranking with (<code>RANK</code>) or without (<code>DENSE_RANK</code>) gaps on ties
<code>LAG()</code> / <code>LEAD()</code>	Value from a previous/next row
<code>SUM()/AVG()</code> <code>OVER (...)</code>	Running totals, moving averages

```
SELECT
  customer_id, order_date, revenue,
  SUM(revenue) OVER (PARTITION BY customer_id ORDER BY order_date) AS running_total
FROM fact_sales;
```

 **Business case:** "Show me each customer's cumulative lifetime spend over time" — the classic running-total window function question, and one of the most common SQL interview questions.

Exercise: For each product, rank its sales by revenue within its category.

► Solution

4.7 Performance, Indexes & Optimization

 **Common mistakes** - `SELECT *` on a wide warehouse table when you need 3 columns — reads far more data than necessary. - Filtering on a function applied to a column (`WHERE YEAR(order_date) = 2026`) — this often prevents the database from using an index. Prefer `WHERE order_date ≥ '2026-01-01' AND order_date < '2027-01-01'` . - Joining on unindexed columns for large tables — check with your Data Engineer if a hot join path is slow.

 **Best practices** - Filter as early as possible (`WHERE` before `JOIN` ing more tables than necessary). - Read the query's **execution plan** (`EXPLAIN` / `EXPLAIN ANALYZE`) when a query is unexpectedly slow — it tells you whether it's scanning a full table or using an index. - Aggregate at the coarsest level the question actually needs — don't pull row-level data into Python just to sum it there.

Chapter Summary

- `SELECT/WHERE/ORDER BY` → `GROUP BY/HAVING` → `JOIN` are the daily-driver 80% of analyst SQL.
- CTEs make multi-step logic readable; Views expose that logic as reusable, trusted building blocks.
- Window functions solve "compare this row to other rows" problems without collapsing your result set — rankings, running totals, period-over-period change.

- Query performance is mostly about filtering early, avoiding functions on filtered columns, and reading the execution plan when something's slow.

Full exercise sets (warm-up → challenge → mini project) live in [../exercises/04_SQL/](#).

Next: [05. Python](#) → — where SQL output becomes cleaning, EDA, and feature engineering.

05 • Python

Part 5 of *The Complete Data Analyst & Data Science Roadmap* · Previous: [04. SQL](#)

 **What you'll be able to do after this chapter** Take a SQL query result and do everything SQL is bad at: deep cleaning, exploratory analysis, feature engineering, and visualization — organized like a professional project, not a scratch notebook.

5.1 Where Python fits (and where it doesn't)

 **Callout** Don't pull millions of rows into Python to do what SQL already does well (aggregation, filtering, joining). Use SQL to get the warehouse to do the heavy lifting, then bring the *result* into Python for what SQL can't do well: statistical modeling, complex cleaning logic, and rich visualization.

```
flowchart LR
    DW[(Data Warehouse)] -->|SQL query, aggregated| DF[Pandas DataFrame]
    DF --> Clean[Cleaning]
    Clean --> EDA[EDA]
    EDA --> FE[Feature Engineering]
    FE --> Viz[Visualization / Stats]
    FE --> ML[Machine Learning\Chapter 08]
```

5.2 Pandas & NumPy

```
import pandas as pd
import numpy as np

df = pd.read_sql("SELECT * FROM vw_monthly_category_revenue", conn)
df.head()
df.info()
df.describe()
```

NumPy underlies Pandas — you'll use it directly for vectorized math and handling NaN:

```
df['revenue_log'] = np.log1p(df['revenue'])
```


 **Real company example** Analysts at companies like **Rappi** or **Amazon** rarely load raw multi-million-row tables into Pandas — they aggregate in SQL first (see [04.7](#)), then bring a few thousand summarized rows into Python for the analysis SQL can't express.

5.3 Cleaning

```
df = df.drop_duplicates()
df['email'] = df['email'].str.strip().str.lower()
df['signup_date'] = pd.to_datetime(df['signup_date'], errors='coerce')
df['revenue'] = df['revenue'].fillna(0)
```

 **Common mistake** Silently dropping rows with `dropna()` without checking *how much* data you're losing or *why* it's missing. Always check `df.isna().sum()` first and understand whether missingness is random or meaningful (e.g., "revenue is null" might mean "order was cancelled," not "data error").

5.4 Exploratory Data Analysis (EDA)

```
df['category'].value_counts()
df.groupby('category')['revenue'].agg(['sum', 'mean', 'count'])
df.corr(numeric_only=True)
```

EDA answers: what does this data actually look like, are there outliers, are there surprises, does anything contradict what stakeholders assume?

5.5 Feature Engineering

```
df['order_month'] = df['order_date'].dt.to_period('M')
df['is_weekend'] = df['order_date'].dt.dayofweek ≥ 5
df['revenue_per_unit'] = df['revenue'] / df['quantity']
```

Feature engineering is where domain knowledge from the business becomes a column a model or a chart can use directly — the bridge into [Chapter 08 \(Machine Learning\)](#).

5.6 Visualization & Statistics

```
import matplotlib.pyplot as plt
import seaborn as sns

sns.lineplot(data=df, x='order_month', y='revenue', hue='category')
plt.title('Monthly Revenue by Category')
plt.show()
```

Pair every chart with the statistic that backs it up — a mean without a sense of spread (std dev, IQR) can mislead a stakeholder.

✅ **Best practice** Always ask "compared to what?" A number alone ("revenue was \$2M last month") means nothing without a comparison — month-over-month, year-over-year, or against a target.

5.7 Project organization, logging, virtual environments

```
project/
├─ data/
├─ notebooks/
├─ src/
│   ├─ clean.py
│   ├─ features.py
│   └─ analysis.py
├─ requirements.txt
├─ .gitignore
└─ README.md
```

```
python -m venv .venv
source .venv/bin/activate      # or .venv\Scripts\activate on Windows
pip install pandas numpy matplotlib seaborn
pip freeze > requirements.txt
```

```
import logging
logging.basicConfig(level=logging.INFO)
logger = logging.getLogger(__name__)
logger.info(f"Loaded {len(df)} rows after cleaning")
```

✅ **Best practice** A **virtual environment** per project + a committed `requirements.txt` is what separates a "script that ran on my machine once" from a project a teammate can actually reproduce. This is exactly the habit you'll lean on for the [10 portfolio projects](#).

Chapter Summary

- Let SQL aggregate; let Python clean, explore, engineer features, and visualize.
- Always quantify missingness before dropping data, and always compare numbers to something.
- Structure every project with `venv` + `requirements.txt` + logging from day one — it's a five-minute habit that makes your work reproducible.

Exercises live in [../exercises/05_Python/](#).

Next: [06. Power BI](#) → — turning this analysis into a dashboard stakeholders actually use.

06 • Power BI

Part 6 of *The Complete Data Analyst & Data Science Roadmap* · Previous: [05. Python](#)

 **What you'll be able to do after this chapter** Build a real dashboard from zero: load and model data, write DAX measures, and publish something a Product Manager or exec would actually trust.

6.1 Power Query

Power Query is Power BI's data-loading and shaping layer (Get Data → Transform). Think of it as a GUI front-end for many of the cleaning steps you did in [Chapter 05](#), except here they compile into a reusable, refreshable pipeline.

 **Best practice** Do heavy transformation upstream (SQL/dbt in the warehouse) and keep Power Query for light shaping — renaming, type-casting, simple filters. A model that leans on the warehouse for logic is easier to debug and much faster to refresh.

6.2 Data Modeling & Relationships

Power BI wants your data modeled the same way you learned in [Chapter 03](#): a **Star Schema** with one fact table and several dimension tables, connected by one-to-many relationships.

```
erDiagram
    FACT_SALES }o--|| DIM_DATE : ""
    FACT_SALES }o--|| DIM_PRODUCT : ""
    FACT_SALES }o--|| DIM_CUSTOMER : ""
```

 **Common mistake** Importing flat, wide, pre-joined tables from a spreadsheet instead of a proper star schema. It works for a tiny dataset, then breaks down fast — duplicated dimension rows, wrong aggregations, and DAX that's needlessly complicated. Model it right from the start.

6.3 DAX, Measures, KPIs

DAX (Data Analysis Expressions) is Power BI's formula language. The single most important habit: write **Measures**, not **Calculated Columns**, for anything you aggregate.

```
Total Revenue = SUM(fact_sales[revenue])

Revenue MoM % =
VAR CurrentMonth = [Total Revenue]
VAR PreviousMonth =
    CALCULATE([Total Revenue], DATEADD(dim_date[full_date], -1, MONTH))
RETURN
    DIVIDE(CurrentMonth - PreviousMonth, PreviousMonth)
```

	Calculated Column	Measure
Computed	Row by row, stored in the model	On the fly, based on filter context
Use for	Static per-row attributes (e.g., <code>Full Name</code>)	KPIs, aggregations, anything that changes with filters
Cost	Increases model size	Computed at query time — no storage cost

 **Business case:** An exec filters the dashboard to "Region = LatAm, Q2." A well-written **Measure** for `Total Revenue` automatically recalculates for that filter context — a calculated column would not.

6.4 Dashboards, Bookmarks, Drill Through

- **Dashboard/report design:** lead with the KPI, support with trend, allow drill-down — don't bury the answer under five charts of context.
- **Bookmarks:** save a specific filter/view state, used to build guided "buttons" that switch report views.
- **Drill Through:** let a user right-click a data point (e.g., a region) and jump to a detail page filtered to just that region.

 **Best practice** Design for the "5-second read": someone glancing at your dashboard mid-meeting should get the headline number and its direction (up/down vs. target) without hunting.

6.5 Deployment, Publishing, Best Practices

```
flowchart LR
    PBIX[Power BI Desktop\n.pbix file] -->|Publish| Service[Power BI Service]
```

```
Service →|Scheduled Refresh| DW[(Data Warehouse)]  
Service →|Share / Embed| Stakeholders[Stakeholders]
```

✅ **Best practices checklist** - Set up **scheduled refresh** so the dashboard is never showing stale data. - Use **Row-Level Security (RLS)** if different regions/teams should only see their own data. - Document your measures (Power BI supports descriptions) so the next analyst doesn't reverse-engineer your DAX. - Version your `.pbix` files and keep a changelog for major dashboard redesigns, same discipline as `../CHANGELOG.md` for this handbook.

Chapter Summary

- Power Query shapes data lightly; heavy transformation belongs upstream in SQL.
- Model in a Star Schema — it's the same discipline from Chapter 03, now inside Power BI.
- Write **Measures**, not Calculated Columns, for KPIs — they respect filter context.
- Publish with scheduled refresh, RLS where needed, and documentation, so the dashboard survives without you in the room.

Exercises live in `../exercises/06_Power_BI/`.

Next: [07. Professional Projects](#) → — combining SQL, Python, and Power BI into a complete, presentable project.

07 · Professional Projects

Part 7 of *The Complete Data Analyst & Data Science Roadmap* · Previous: [06. Power BI](#)

 **What you'll be able to do after this chapter** Scope, structure, and deliver a project the way it actually happens inside a company — not a Kaggle notebook, but a business problem taken from question to executive summary.

7.1 The methodology

Every project in [../projects/](#) follows the same seven-step arc — this is also, not coincidentally, how real analyst work gets scoped at a company:

flowchart LR

```
A[1. Business Problem] → B[2. Data & Schema]
B → C[3. Questions to Answer]
C → D[4. SQL]
D → E[5. Python]
E → F[6. Power BI]
F → G[7. Executive Summary]
```

1. **Business Problem** — one paragraph, written like a stakeholder actually asked it ("Why is churn rising in the Northeast?"), not like a dataset description.
2. **Database Diagram** — the schema you're working with, drawn like [Chapter 03](#).
3. **Questions to Answer** — 4–6 concrete questions the business problem breaks down into.
4. **SQL** — extraction and aggregation per [Chapter 04](#).
5. **Python** — cleaning, EDA, feature engineering per [Chapter 05](#).
6. **Power BI** — the dashboard stakeholders will actually open, per [Chapter 06](#).
7. **Executive Summary** — one page: what you found, what you recommend, what you'd need to go further.

 **Best practice** Write the Executive Summary **first**, as a hypothesis, before touching data. It forces you to state what a good answer would even look like — then the analysis either confirms it, kills it, or complicates it. That's a much stronger project narrative than "I explored the data and found some things."

 **Common mistake** Leading a project with the tools ("I used Python and Power BI to analyze retail data") instead of the business problem. Recruiters and hiring managers skim dozens of these

— the ones that read like a business case study, not a tools showcase, stand out.

7.2 Project folder structure

Every project in `../projects/` uses this structure:

```
projects/<NN_Project_Name>/
├─ README.md           # business problem, objectives, architecture, workflow, conclusions
├─ data/
├─ sql/
├─ python/
├─ powerbi/
├─ diagrams/
└─ presentation/
```

7.3 The 10 projects

#	Project	Difficulty	Primary skills
01	Retail Sales Analytics	Beginner	SQL, Power BI
02	E-commerce Customer Churn	Intermediate	SQL, Python
03	Adventure Works BI	Intermediate	SQL, Power BI
04	Northwind Operations	Beginner	SQL
05	Credit Card Fraud Detection	Advanced	Python, ML
06	HR Attrition Analytics	Intermediate	SQL, Python, Power BI
07	Marketing Campaign Performance	Intermediate	SQL, Power BI
08	Financial Forecasting	Advanced	Python, Time Series
09	Product Recommendation Engine	Advanced	Python, ML
10	Capstone End-to-End Pipeline	Advanced	SQL, Python, Power BI, ML

Each project's own `README.md` (in its folder under `../projects/`) follows the 7-step structure from §7.1 in full.

Chapter Summary

- A real project starts with a business problem stated the way a stakeholder would ask it, not a dataset description.

- Structure: Business Problem → Schema → Questions → SQL → Python → Power BI → Executive Summary.
- Write the Executive Summary as a hypothesis first — it sharpens the whole analysis.
- The 10 projects in [../projects/](#) apply this methodology end to end, increasing in difficulty.

Next: [08. Machine Learning](#) → — for the projects (05, 08, 09, 10) that go beyond SQL/BI into predictive modeling.

08 • Machine Learning

Part 8 of *The Complete Data Analyst & Data Science Roadmap* · Previous: [07. Professional Projects](#)

 **What you'll be able to do after this chapter** Frame a business problem as a modeling problem, build a proper evaluation pipeline, and understand what it takes to get a model into production — the Data Analyst → Data Scientist bridge from [01.4](#).

8.1 The problem types

Type	Predicts	Example
Regression	A continuous number	Forecast next month's revenue
Classification	A category	Will this customer churn? (yes/no)
Clustering	Ungrouped structure	Segment customers by behavior, no labels given
Recommendation	Ranked relevance	"Customers who bought X also bought Y"
Time Series	Future values over time	Daily demand forecasting
NLP	Structure/meaning from text	Classify support tickets by topic, sentiment of reviews

 **Real company example** A **Credit Card Fraud Detection** system (see [Project 05](#)) is a classification problem with extreme class imbalance — fraud is maybe 0.1% of transactions. Framing this correctly (and choosing the right metric) matters more than which algorithm you pick.

8.2 Feature Engineering (recap + modeling lens)

Everything from [05.5](#) applies — now with a modeling goal: does this feature actually help the model separate classes / predict the number, or is it noise the model will overfit to?

```
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.compose import ColumnTransformer

preprocess = ColumnTransformer([
    ('num', StandardScaler(), ['revenue_per_unit', 'order_count']),
```

```
('cat', OneHotEncoder(handle_unknown='ignore'), ['category', 'region'])),
])
```

8.3 Pipelines & Cross-Validation

```
from sklearn.pipeline import Pipeline
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import cross_val_score, train_test_split

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, stratify=y, random_state=42)

pipe = Pipeline([
    ('preprocess', preprocess),
    ('model', RandomForestClassifier(random_state=42)),
])

scores = cross_val_score(pipe, X_train, y_train, cv=5, scoring='f1')
pipe.fit(X_train, y_train)
```

✅ **Best practice** Always wrap preprocessing + model in a single `Pipeline`, and always cross-validate. A model that looks great on one train/test split and terrible on another isn't ready — this is the #1 way junior projects overstate results.

8.4 Model Evaluation

Metric	Use when
RMSE / MAE	Regression — average error magnitude
Accuracy	Classification, only when classes are balanced
Precision / Recall / F1	Classification with imbalance (fraud, churn)
ROC-AUC	Ranking how well the model separates classes across thresholds

⚠️ **Common mistake** Reporting 99% accuracy on a fraud model where fraud is 0.1% of the data — a model that predicts "not fraud" for everything gets 99.9% accuracy while being completely useless. This is exactly why **Precision/Recall** exist.

8.5 Hyperparameter Optimization

```
from sklearn.model_selection import GridSearchCV

param_grid = {'model__n_estimators': [100, 300], 'model__max_depth': [5, 10, None]}
grid = GridSearchCV(pipe, param_grid, cv=5, scoring='f1')
grid.fit(X_train, y_train)
print(grid.best_params_)
```

8.6 Deployment (the ML Engineer handoff)

```
flowchart LR
    NB[Notebook: trained model] --> S[Serialize[Serialize\n.pkl / MLflow]]
    S --> API[API[Wrap in API\nFastAPI / Flask]]
    API --> Prod[Prod[Production service]]
    Prod --> M[Monitor[Monitor: drift, latency, accuracy]]
    M -->|triggers| R[Retrain[Retraining pipeline]]
```

 **Real company example** A Data Scientist's churn model doesn't create business value sitting in a notebook. An **ML Engineer** wraps it behind an API, the product calls it when deciding who gets a retention offer, and the model is monitored for **drift** — when live data starts looking different from training data, accuracy silently degrades. This is the role boundary from [01.2](#).

Chapter Summary

- Correctly frame the business problem as regression / classification / clustering / recsys / time series / NLP before picking an algorithm.
- Always use a Pipeline + cross-validation — never trust a single train/test split.
- Match the metric to the problem: imbalanced classification needs Precision/Recall/F1, not accuracy.
- Deployment is a distinct discipline (ML Engineering) — training a good model is necessary but not sufficient.

Exercises live in [../exercises/08_Machine_Learning/](#). Applied end-to-end in [Projects 05, 08, 09, 10](#).

Next: [09. Portfolio & Career](#) → — turning everything you've built into a job.

09 · Portfolio & Career

Part 9 of *The Complete Data Analyst & Data Science Roadmap* · Previous: [08. Machine Learning](#)

💡 **What you'll be able to do after this chapter** Package everything from Chapters 1–8 into a portfolio and job search that actually converts — Git hygiene, a resume that reads like impact instead of tasks, interview prep, and a freelancing on-ramp if you want one.

9.1 Git & GitHub

```
git init
git add .
git commit -m "Add customer churn analysis: SQL extraction + Python EDA"
git remote add origin <your-repo-url>
git push -u origin main
```

✅ **Best practice** Commit in meaningful chunks with messages that describe *why*, not "update file" — a hiring manager skimming your commit history is silently evaluating how you'll work on their team. This is the same discipline this handbook's own [CHANGELOG.md](#) models.

9.2 README & Project Structure

Every project ([Chapter 07](#)) needs a README that a recruiter can read in 60 seconds and understand:

What was the business problem, what did you do, what did you find. Lead with a screenshot of the dashboard or a key chart — not a wall of text.

⚠️ **Common mistake** A GitHub profile with 15 tutorial-following repos ("Titanic Notebook #1") and zero finished, narrated projects. Three complete projects with a real business framing (per [Chapter 07](#)) beat fifteen unfinished ones, every time.

9.3 LinkedIn & Resume

Resume bullet formula: [Action] + [what you analyzed/built] + [quantified business impact]

Weak	Strong
"Used SQL and Python to analyze sales data"	"Identified a \$180k/year revenue leak in a discount-stacking bug using SQL cohort analysis; recommendation adopted by Finance"
"Built Power BI dashboards"	"Built an executive Power BI dashboard adopted by 3 regional teams, replacing 6 hours/week of manual reporting"

✅ **Best practice** Every bullet should survive the question "so what?" If it doesn't end in an outcome a business person cares about, rewrite it.

9.4 Interview Questions

SQL round (see also [Chapter 04](#)): - Write a query to find the second-highest salary per department. - Explain the difference between `WHERE` and `HAVING`. - Given two tables, write a query to find customers with no orders.

Python round (see also [Chapter 05](#)): - Given a messy DataFrame, walk through your cleaning approach out loud. - Explain `.groupby()` vs. a pivot table — when would you use each?

Case study / business round: - "Weekly active users dropped 10% last week — walk me through how you'd investigate." - Practice structuring your answer: clarify the metric definition → check for data issues → segment (by platform, geography, cohort) → form and test a hypothesis → recommend next steps.

ML round (see also [Chapter 08](#)): - Precision vs. Recall — when do you optimize for one over the other? - How do you know if a model is overfitting?

Track your own applications and questions in the [Notion Interview Tracker](#).

9.5 Freelancing (Fiverr, Upwork)

Freelance data work is a legitimate way to build portfolio *and* income while job-hunting:

- Start with small, well-scoped gigs: "clean and analyze this dataset," "build me a 3-page Power BI dashboard."
- Price by the deliverable, not the hour, once you have 2-3 reviews.
- Every freelance project is a portfolio project — apply the [Chapter 07](#) structure to it too.

⚠️ **Common mistake** Underpricing to win the first gig, then staying underpriced forever. Price low for your first 2-3 reviews only, then raise rates — reviews are the asset you're actually buying at that stage.

9.6 Learning Timelines

Track	Pace	Rough chapter pacing
6 months (intensive, ~25-30 hrs/wk)	Ch.01–03: 2 wks · SQL: 4 wks · Python: 5 wks · Power BI: 3 wks · Projects: 6 wks · ML: 4 wks · Portfolio/Job search: ongoing from month 4	
12 months (balanced, ~12-15 hrs/wk)	Same order, roughly double the time per phase, with 2 projects running in parallel with each new skill	
18 months (part-time, ~6-8 hrs/wk)	Same order, one topic at a time, minimum 1 project shipped per quarter	

Weekly/daily schedule templates are in the [Notion Learning Calendar](#).

9.7 Senior Engineer Advice

 **From the trenches** - "The best analysts I've hired weren't the best at SQL — they were the best at asking 'is this the right question?' before writing any query." - "Your first six months on the job, over-communicate assumptions. A wrong number nobody questioned is worse than a slow number everyone trusts." - "Don't wait for permission to be curious about a metric that looks off. That instinct is the actual job."

Chapter Summary

- Git/GitHub hygiene and a README that leads with business impact are part of the deliverable, not an afterthought.
- Resume bullets should survive "so what?" — quantify business impact, not just tools used.
- Prepare for four interview lanes: SQL, Python, business case, and (if relevant) ML.
- Freelancing is a legitimate parallel path — treat every gig like a portfolio project.
- Pick a timeline (6/12/18 months) that matches your real available hours, not your ambition.

This completes the core handbook. Return to the [Table of Contents](#) · Continue to the [10 portfolio projects](#) to apply everything.